Discovering Where Episodes Begin: How OmniReset Generates Its Reset-State Datasets

Tri Wahyu Guntara
Tutorial notes on the UWLab OmniReset code base

Abstract: A reinforcement-learning policy for peg insertion can only become competent in the situations it has been placed in. OmniReset therefore begins each training episode from a library of *reset states*, complete snapshots of the scene that physics has already accepted as stable, organised into four families: the object anywhere with the hand anywhere, the object resting with the hand grasping it, the object held in the air, and the object partly inserted with the hand on it. This tutorial explains how that library is produced. We define what a reset state is and how it is stored, describe the two auxiliary datasets of grasps and partial assemblies on which the grasped families depend, walk through one recording attempt from the reset events to the export decision, derive the inverse-kinematics placement of the hand on a sampled grasp, and examine the success test clause by clause. We close with the commands that run the generator, the failure modes that motivated each clause, and exercises for the reader. Every statement is tied to the implementation on the `feature/forge-omnireset` branch.

Keywords: Reset-state distributions, Reinforcement learning, Simulation, Grasp sampling, Inverse kinematics

1 Why Reset States Exist

A reinforcement-learning agent learns from episodes, and every episode begins somewhere. If every episode of a peg-insertion task began with the peg lying on the table and the hand hovering far away, the policy would learn to approach, grasp, lift, and insert in that one order. It would never practise recovering from a peg that is already half inserted, or from a grasp that is slightly off centre, because it would never start there.

OmniReset [1] addresses this by beginning episodes from a large library of reset states. A reset state is a complete snapshot of the scene, a configuration that physics has already accepted as stable. The library is organised into the four families of Table 1, each describing where the object is and what the hand is doing. During training the environment draws each new episode's starting state from one of the four files, with probability one quarter each. Evaluation uses the first family alone, so that a reported success rate always means "from scratch". Because every state in the library has been simulated and checked before it was saved, the policy never starts inside a wall, with the peg floating, or with the arm folded through itself.

The central idea is that reset states are not drawn from a formula. They are *discovered*. Thousands of simulated environments guess a configuration, hold it for two seconds under physics, and keep only the guesses that survive a strict test. The dataset is the collection of survivors.

2 What a Reset State Is

Formally, a reset state is the output of the scene's own state query, taken relative to the environment origin. For one environment it contains, for every articulation (here only the robot), the root pose $(\boldsymbol{p}, \boldsymbol{q}) \in \mathbb{R}^7$, the root velocity in $\mathbb{R}^6$, one position per joint, and one velocity per joint; and for every

Table 1: The four families of reset states. The identifiers are the strings that appear in the task names and in the dataset file names.

Family	Object	Hand	What it teaches
`ObjectAnywhere-EEAnywhere`	Dropped from a random pose, lands where it may	Random pose in the workspace, fingers open or closed	Approaching and grasping from scratch
`ObjectResting-EEGrasped`	Resting on the table, taken from the first family	Closed on the object at a sampled grasp	Lifting and carrying a grasped object
`ObjectAnywhere-EEGrasped`	Held in the air	Closed on the object at a sampled grasp	Transport and alignment above the hole
`ObjectPartiallyAssembledEEGrasped`	Partly inserted, taken from the partial-assembly dataset	Closed on the object at a sampled grasp	Finishing an insertion, or recovering from a jam

rigid object (the insertive object, the receptive object, and the table), the root pose in $\mathbb{R}^7$ and the root velocity in $\mathbb{R}^6$.

Everything that physics needs to continue the simulation is included. Nothing about the controller is included, and this matters for the grasped families. The recorded finger positions already sit at the grasp, but the clamping force of a gripper comes from the controller commanding a position *past* the contact. Restoring the state therefore restores the geometry of a grasp but not its squeeze. The first gripper action of the policy decides whether the object stays in the hand.

Where the file lives and how it is named. Each object pair gets a directory named after the two assets. The name is formed from the parent folder of each USD file, sorted alphabetically and joined by two underscores, so the peg and its hole become `Peg__PegHole`. Inside that directory the four families are stored as `resets_<Family>.pt`, written by `torch.save`:

```
Datasets/OmniReset_g1d/
|-- Grasps/
|   `-- Peg/grasps.pt                 gripper pose in the object frame, finger joints
`-- Resets/
    `-- Peg__PegHole/
        |-- partial_assemblies.pt      object pose in the hole frame, partly inserted
        |-- resets_ObjectAnywhereEEAnywhere.pt
        |-- resets_ObjectRestingEEGrasped.pt
        |-- resets_ObjectAnywhereEEGrasped.pt
        `-- resets_ObjectPartiallyAssembledEEGrasped.pt
```

A reset file is a nested dictionary. The top key is `initial_state`; below it come `articulation` and `rigid_object`, then the asset name, and finally the quantity. Each leaf is a list with one tensor per saved state, and the loader counts states by the length of the robot's `joint_position` list. Every file also carries the entry `quat_convention: "xyzw"`. The loader refuses a file without it, because datasets recorded under Isaac Lab 2 used the other quaternion order, and loading one silently rotates every pose by 180° about the x axis.

3 The Three Datasets and Their Dependencies

Reset states are the last of three datasets. The other two provide the raw material for the grasped and partially assembled families. Figure 1 shows what depends on what, and it also fixes the order in which the generator must run.

3.1 Partial Assemblies

The partial-assembly dataset answers the question "what does a half-finished insertion look like?" The procedure places the insertive object exactly at its assembled pose, which the asset metadata provides as an offset relative to the receptive object. It then applies a tiny random force and torque to

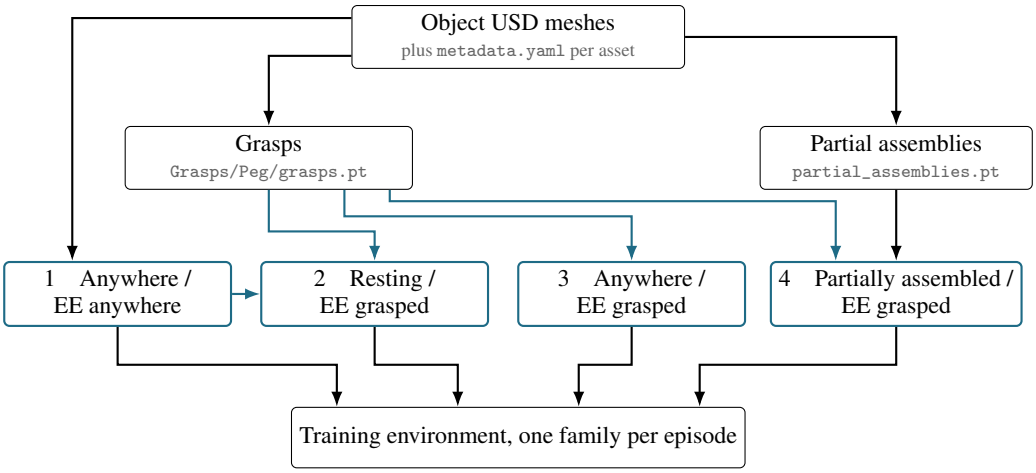


Figure 1: Dependency graph of the three datasets and the four reset families. Grasps (blue arrows) feed every grasped family. Family 2 additionally draws its object poses from family 1, so family 1 must be recorded first. The pipeline script runs partial assemblies, then grasps, then the four families in numerical order.

the object on every physics step, so that the object jiggles out of the hole along whatever directions the geometry allows. At each physics step the pose of the insertive object expressed in the frame of the receptive object is logged. A pose is kept when the objects are collision-free at that instant and when it differs from every pose already kept by more than a position and an orientation threshold. The result is a set of relative poses $(\boldsymbol{p}_{\text{ins}}^{\text{rec}}, \boldsymbol{q}_{\text{ins}}^{\text{rec}})$ that span the insertion path. The file does not store a robot state. It stores only where the object may sit in the hole.

3.2 Grasps

The grasp dataset answers "how can this gripper hold this object?" It is recorded with a gripper alone, anchored to the world, and the object floating in front of it. Candidate grasps are generated geometrically by antipodal sampling on the object mesh [2].

1. Sample points on the mesh surface, together with their outward normals. A share of the samples is taken from the highest points with upward-facing normals; the rest are uniform over the surface. For the Dex1 gripper this share is one half, so that a peg lying on its side also receives grasps across its diameter.
2. From each sample, cast a ray into the object along the negative normal. The farthest hit that lies within the gripper's maximum aperture becomes the opposing contact. The segment between the two contacts is the *grasp axis*, and its midpoint is the *grasp centre*.
3. Align the gripper's closing axis with the grasp axis, rotate about the gripper's approach direction through sixteen equally spaced angles, and for each angle retreat the gripper along its approach direction through thirty-two standoff distances between the finger offset and the finger offset plus the mesh diagonal.

Every candidate is a rigid transform $T_{\text{grip}}^{\text{obj}}$ of the gripper in the object frame. At each reset the environment opens the fingers, teleports the anchored gripper to a random candidate around the object, and commands the fingers closed. Gravity is switched on after one second and small disturbance forces are applied. The episode lasts four seconds. A candidate becomes a recorded grasp when, at the end, the object has moved less than a quarter metre from where it started, is still above half the spawn height, is not interpenetrating the gripper, has settled, and the fingers have travelled at least ten percent of the way from open to closed. The last clause rejects an object that merely rests across fully open fingers. The recorder saves the gripper pose in the object frame and the finger joint positions.

Table 2: What the reset events do in each family. "IK" is the iterative inverse-kinematics placement of Section 6.

Family	Insertive object	End effector and fingers
1 Anywhere / EE anywhere	Uniform pose above the table, up to 25 cm high, any orientation	IK to a uniform pose in the workspace, pointing roughly down. Fingers follow the gripper command, drawn at random to be open or closed.
2 Resting / EE grasped	A state sampled from the *family 1* file, so the object is already resting as physics left it	IK to a sampled grasp on the object, perturbed by up to 2 cm and 11°. Fingers written to the recorded grasp positions.
3 Anywhere / EE grasped	Uniform pose above the table, as in family 1	IK to a sampled grasp on the object, no perturbation. Fingers written to the recorded grasp positions.
4 Partially assembled / EE grasped	A relative pose sampled from the partial-assembly file, composed with the hole's pose	IK to a sampled grasp, perturbed by up to 1 cm and 6°. Fingers written to the recorded grasp positions.

3.3 Reset States

The reset-state generator combines these two files with a full robot and a table. The remainder of these notes is about it.

4 The Recording Environment

Reset states are recorded by an ordinary Isaac Lab [3] manager-based environment that has no observations and no rewards. It has a scene, a set of reset events, a set of termination terms, and a recorder. The script that drives it is `scripts_v2/tools/record_reset_states.py`. Each family is a separate registered task, for example `OmniReset-G1dDex1-ObjectRestingEEGrasped-v0`.

The scene. The scene holds the robot, the insertive object, the receptive object, a table, and a ground plane. The insertive object is a dynamic rigid body with a mass of one gram, so that it does not drag the arm down while held. The receptive object is kinematic, so it never moves. Friction on every body is fixed at a low value by a start-up event. For the G1-D the table is a plain block with the Vention table's footprint and top height.

Timing. Physics runs at 120 Hz and the environment steps every twelve physics steps, so the control rate is 10 Hz. The episode length is two seconds. One recording attempt therefore lasts

$$N_{\text{steps}} = \frac{T_{\text{episode}}}{\Delta t \, d} = \frac{2.0 \text{ s}}{(1/120 \text{ s}) \times 12} = 20 \text{ control steps} = 240 \text{ physics steps}. \tag{1}$$

Reset events, by family. When an environment resets, its event terms run in declaration order. Every family first restores the scene to its default and places the receptive object at a random pose on the table within a small yaw range. What happens next depends on the family (Table 2). Notice that family 2 loads family 1's file through the same `MultiResetManager` that training uses. The dependency is literal: if family 1 contains a peg that rolled off the table, family 2 will attempt grasps on the floor.

Termination terms. Four terms can end an episode. Their names matter, because the recorder looks for a term called `success`.

`time_out` ends the episode after twenty steps. Nothing is saved.

`abnormal_robot` ends the episode at once if any joint moves faster than twice its velocity limit, which indicates a solver explosion.

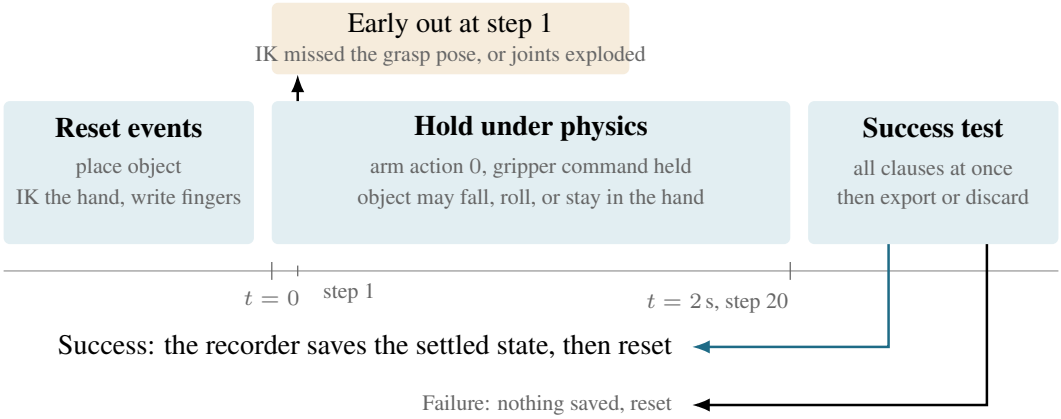


Figure 2: Timeline of one recording attempt in one environment. Thousands of these run in parallel, each on its own schedule, and the script counts exported episodes until it reaches the requested number.

`grasp_placement_failed` applies to the grasped families only. It ends the episode at the first step if the IK placement ended more than 1 cm or 10° from the sampled grasp pose. Such an attempt can never pass the success test, so holding it for two seconds would only cost time. Families without a grasp event never trigger it.

`success` is evaluated every step but can only be true on the final step, as Section 7 explains. When it is true the episode is exported.

Actions during the hold. The arm action is zero. The operational-space controller therefore holds the end effector where the reset placed it, with its own stiffness and damping. The gripper action is the last element of the action vector. For any family whose name contains `EEGrasped` it is held at -1, the close command, for the whole run. For the other families it is drawn at random from $\{-1, +1\}$ per environment and redrawn whenever that environment resets, so the first family contains both open and closed hands.

The recorder. The recorder manager is configured with a single term that fires just before an environment resets. The term calls the scene's state query for the environments that are about to reset and returns it under the key `initial_state`. The export mode is "succeeded only". Isaac Lab's reset routine reads the `success` termination term for the resetting environments, marks their episodes accordingly, and the manager appends only the marked ones to the file. A reset state is therefore the scene *as it stands after the two-second hold*, not as the events placed it. It is already settled.

5 One Episode, Step by Step

Follow one environment through Figure 2.

1. **Reset.** The scene returns to its defaults. The receptive object is placed. The insertive object is placed by the family's rule. For a grasped family, a grasp is drawn, the end-effector target is composed, the iterative IK writes the arm joints, and the finger joints are written to the recorded values. The success term records the end-effector position and each object's position as the reference for the deviation test, and the stability counter is zeroed.
2. **Step 1.** Physics runs twelve substeps. If the IK residual from the reset was too large, or any joint velocity is abnormal, the episode ends here. Otherwise the hold begins.
3. **Steps 2 through 19.** The arm holds. The gripper tracks its command. Whatever is not physically consistent falls, slides, or settles. On every step the success term checks whether all bodies are

nearly at rest and increments a counter, resetting it to zero whenever anything moves. At least five consecutive quiet steps are required at the end.

4. **Step 20.** The time-out flag becomes true, and only now can the success term return true. Every clause of Section 7 is evaluated on the state as it stands.
5. **Export.** If successful, the recorder copies the environment's full state into the dataset file. In both cases the environment resets and the cycle restarts.

The script tracks the recorder's count of exported episodes and stops when it reaches `-num_reset_states`. Because attempts are cheap and failures are common, the yield is best thought of as a rate. If an attempt succeeds with probability p and E environments run in parallel, the expected number of control steps needed to collect n states is

$$\mathbb{E}[\text{steps}] \approx \frac{n}{p\,E} \times 20. \tag{2}$$

At the smoke-test scale used in the repository, 256 environments and 8 states per family, the grasped families succeed in between 0.01 and 0.06 percent of attempts and take between two and seven minutes. The first family succeeds far more often because it asks for much less.

6 Placing the Hand on a Grasp

The grasped families reuse a grasp that was recorded with a bare gripper. The event `reset_end_effector_from_grasp_dataset` must turn that relative pose into joint angles for the whole arm. The computation has three parts.

Composing the target. Write T_B^A for the pose of frame B expressed in frame A. The dataset stores the gripper in the object frame, $T_{\text{grip}}^{\text{obj}}$, and the object has just been placed at $T_{\text{obj}}^{\text{w}}$ in the world. The desired gripper pose in the world is therefore

$$T_{\text{grip}}^{\text{w}} = T_{\text{obj}}^{\text{w}}\, T_{\text{grip}}^{\text{obj}}. \tag{3}$$

The IK solver works in the robot's base frame, so the target is expressed there, and an optional perturbation T_δ, drawn uniformly from the family's `pose_range_b`, is applied on the right, that is, in the gripper's own frame:

$$T_{\text{target}}^{\text{b}} = \left(T_{\text{b}}^{\text{w}}\right)^{-1} T_{\text{grip}}^{\text{w}}\, T_\delta. \tag{4}$$

The event also stores the target back in the object frame, $T_{\text{target}}^{\text{obj}} = (T_{\text{obj}}^{\text{w}})^{-1}\, T_{\text{b}}^{\text{w}}\, T_{\text{target}}^{\text{b}}$, perturbation included. Section 7 compares the end effector against this quantity after the hold, which is how the test knows that the hand is on the object rather than merely near it.

Iterating the inverse kinematics. Isaac Lab's differential IK with damped least squares [4, 5] gives, from the current joint vector $\boldsymbol{q}_k$ and the pose error, a joint target $\boldsymbol{q}_k^*$. Rather than jumping to it, the event takes a quarter step, clamps to the joint limits, and writes the result straight into the simulation without stepping physics:

$$\boldsymbol{q}_{k+1} = \operatorname{clamp}\Big(\boldsymbol{q}_k + \tfrac{1}{4}\,(\boldsymbol{q}_k^* - \boldsymbol{q}_k),\ \boldsymbol{q}_{\min},\ \boldsymbol{q}_{\max}\Big), \qquad k = 0, \dots, 24. \tag{5}$$

Twenty-five iterations at a quarter step each drive the pose error down by a factor of roughly $0.75^{25} \approx 10^{-3}$ whenever the target is reachable. Writing the joint state, rather than commanding it, means that the forward kinematics are refreshed on the next read, so each iteration sees the new pose. The clamp is essential. Without it the wrist roll and yaw were routinely written several radians beyond their limits; PhysX snapped them back on the first physics step, and the hand was thrown away from the object it had been placed on.

Table 3: The clauses of the success test. Clauses marked (G) apply to the grasped families only; the last applies to family 4 only.

Clause	Condition	Why it is there
Time out	Step counter has reached 20	The state must have survived the full hold.
No abnormal gripper	No joint faster than twice its velocity limit	Rejects solver blow-ups.
Approach points down	The gripper approach axis, rotated into the world, has $z < -0.5$, i.e. within 60° of straight down	Keeps hand orientations in the range a top-down insertion policy will see.
Stable	Five consecutive steps with robot joint speeds summing below 5 rad/s and each object's linear speed below 0.1 m/s and angular speed below 1 rad/s	The scene has settled; starting mid-fall would be unfair.
Small deviation	End effector moved less than 10 cm since reset; each object moved less than the family's limit: unbounded for family 1, 1 cm for 2, 5 cm for 3, 2.5 cm for 4	The state must be close to what was asked for. A grasped peg that slid 5 cm was not really grasped.
Above the table	Every body above `pos_z_threshold`, for the G1-D 2 cm below the table top	A peg that rolled off the edge is not a usable start.
Collision free	Three point-cloud distance checks pass	No interpenetration between robot and objects, or between the objects.
Fingers blocked (G)	Every finger's open fraction lies strictly between 0.08 and 0.9	Something is between the fingers, and the hand is not simply open.
Object near the hand (G)	Object origin within 20 cm of the end-effector body	Coarse sanity check.
At the grasp pose (G)	End effector, in the object frame, within 1.5 cm and 15° of the target stored at reset	Tells a hand closed on the peg from one closed on the hole, the table edge, or thin air nearby.
Assembly match (4)	Half of the environments must be within 1.5 times the assembly success threshold, the other half must not be	Keeps a balance of "nearly done" and "still jammed" starts.

Recording the residual. After the last iteration the event measures how far the end effector ended from the target, in position and in angle, and stores the two numbers per environment. The `grasp_placement_failed` termination reads them and discards any attempt beyond 1 cm or 10° at step one. Every twentieth call the event prints the fraction of environments that reached the target together with the median and ninetieth-percentile errors, which is the quickest way to see whether a layout is reachable at all. Finally the finger joints are written to the positions recorded in the grasp. Grasps whose fingers were recorded nearly fully open, more than ninety percent of the way from the close command to the open command, are dropped when the file is loaded, because such a "grasp" was an object lying across open fingers.

Reach is a property of the layout. An offline probe of the G1-D with the Dex1 gripper found that only about 28% of resting-peg and grasp pairs are reachable within 1 cm and 10°, and that neither more iterations nor other random seeds change that number. Reachability falls sharply beyond 36 cm from the robot, because the grasp centre of a top-down Dex1 grasp sits 16 cm below the wrist. The remedy is to move the robot closer to the table or to narrow the spawn ranges in the layout file, not to tune the IK.

7 The Success Test, Clause by Clause

The term `check_reset_state_success` returns the conjunction of several boolean clauses. A state is saved only if every clause holds at the final step. Table 3 lists them in the order the code evaluates them, with the values used for the G1-D peg task.

The open fraction. Several clauses need to know how closed a gripper is without knowing its mechanism. The code reads the open and close command dictionaries of the task's binary gripper

action and, for each commanded joint with position θ, computes

$$f_{\text{open}} = \left| \frac{\theta - \theta_{\text{closed}}}{\theta_{\text{open}} - \theta_{\text{closed}}} \right|, \tag{6}$$

which is zero at the close command and one at the open command. A gripper that closes on nothing reaches $f_{\text{open}} \approx 0$; one that closes on a 2 cm peg stops partway; one that was never commanded closed sits at $f_{\text{open}} \approx 1$. The same quantity, with the roles reversed, is used by the grasp recorder's "fingers travelled at least ten percent" clause.

The collision analyzer. Interpenetration is measured, not inferred from contacts. At start-up the analyzer samples 1024 points on the surface of each link of the asset being checked and builds a signed-distance representation of each obstacle from its collision meshes. On every call it transforms the point cloud into each obstacle's frame and queries the signed distance of every point. The clause holds when the smallest distance is at or above a per-pair minimum: half a millimetre of allowed penetration between robot and insertive object, none between robot and receptive object, and one millimetre between the two objects. The small negative allowances absorb solver tolerance in a grasp without accepting a finger inside a peg.

Reading the funnel. With the optional `collect_funnel` parameter the term counts, over all environments whose hold just ended, how many passed each clause, and the debug tool prints the table. It is the first thing to look at when a family yields nothing, because it says which clause is doing the rejecting. In the resting family, before the fixes of Section 9, 73% of holds failed the deviation clause alone, and the grasp-pose clause then removed almost everything that remained.

8 Running the Generator

The generator runs inside the project's Docker image. Locally the wrapper `docker/forge/run_local.sh` mounts the working copy and runs any command; the cluster pipeline `docker/forge/g1d_pipeline.sh` runs the same commands in its `datasets` stage, in the order fixed by Figure 1:

```
# 1. partial assemblies for the object pair
python scripts_v2/tools/record_partial_assemblies.py --task OmniReset-PartialAssemblies-v0 \
    --num_envs 10 --num_trajectories 10 --dataset_dir $DATA_DIR \
    env.scene.insertive_object=peg env.scene.receptive_object=peghole

# 2. grasps on the insertive object with the Dex1 gripper
python scripts_v2/tools/record_grasps.py --task OmniReset-Dex1-GraspSampling-v0 \
    --num_envs 8192 --num_grasps 1000 --dataset_dir $DATA_DIR \
    --gripper_body_name right_dex1_base_link env.scene.object=peg

# 3. the four families, in this order
for RT in ObjectAnywhereEEAnywhere ObjectRestingEEGrasped \
          ObjectAnywhereEEGrasped ObjectPartiallyAssembledEEGrasped; do
  python scripts_v2/tools/record_reset_states.py --task OmniReset-G1dDex1-${RT}-v0 \
      --num_envs 2048 --num_reset_states 10000 --dataset_dir $DATA_DIR \
      env.scene.insertive_object=peg env.scene.receptive_object=peghole
done
```

The pipeline's defaults are 2048 environments and 10 000 states per family; the environment variables `RESET_ENVS`, `NUM_RESETS`, `GRASP_ENVS` and `NUM_GRASPS` override them. The stage is cached on the shared disk and skipped when the last family's file already exists, so a regeneration after a code change needs `FORCE=1`. The script itself does three things for the user: it sets the seed to `None`, so that every run explores different configurations; it infers the family from the task name and derives the pair directory from the two USD paths, so that the output lands where the training environment will look for it; and it fixes the gripper command, closed for any `EEGrasped` task and random otherwise.

Three tools look at a finished file. `visualize_reset_states.py` cycles through states in a window. `render_reset_states.py` writes one image per state headlessly and, with `-settle_steps 240 -gripper close`, shows the same state after a further two seconds with the task's own controller holding the arm and the gripper commanded closed, which is the fairest picture of what a policy will

inherit. `debug_reset_env.py -funnel -close_gripper` runs the recording environment and prints the clause-by-clause funnel.

9 What Goes Wrong, and What It Looks Like

Each clause of the success test exists because some plausible-looking state passed without it. The following failures were all observed on the G1-D peg task.

The hand is closed on the hole, or on the table, and the peg lies nearby. The older test required only that the fingers be stopped and the peg within 20 cm. A hand that closed on the rim of the hole satisfied both. The grasp-pose clause, which compares the end effector with the stored target in the object frame, is the fix.

An open hand hovers above a partly inserted peg. The recorder once commanded the gripper closed only for two families by name, and the partially assembled family was not among them. Half of its environments held with open fingers, and an open hand trivially satisfies "not fully closed". The recorder now closes for every grasped family, and the test rejects a hand more than ninety percent open.

The hand jumps away on the first physics step. IK solutions beyond the joint limits were snapped back by PhysX. Clamping each iterate, Eq. (5), removed most of the 10 cm deviations.

No grasp fits a peg lying on its side. With all surface samples biased to the top of the mesh, every grasp approached along the peg's axis. A lying peg then needed a horizontal approach, which the "points down" clause rejects. Mixing in uniform surface samples produced grasps across the diameter.

A peg on the floor. Family 1's original height threshold was inherited from the UR5e, whose mounting plate sits near the origin. For the floor-standing G1-D it had to be raised to just below the table top.

Everything is slow. Most grasped attempts are unreachable and were held for the full two seconds. Ending them at step one made recording three to six times faster.

10 Exercises

Solutions are given in Appendix A.

E1. Why can the `success` term never be true before the last step, even though it is evaluated every step?

E2. A recorded grasped state is restored at the start of a training episode and the peg immediately falls. Nothing is wrong with the dataset. Explain.

E3. Family 2 yields nothing. The funnel shows almost every attempt failing the deviation clause for the robot. Where do you look first?

E4. Why is the perturbation T_δ in Eq. (4) applied in the gripper frame rather than in the object frame?

E5. Equation (2) says that doubling the number of environments halves the time. When does that stop being true?

11 Limitations

These notes describe the generator as implemented on the `feature/forge-omnireset` branch of the UWLab repository on 27 October 2026 and may drift from later revisions. The thresholds quoted are those of the G1-D peg task; the UR5e configurations share the code but differ in several numbers, most visibly the height threshold. The yields quoted were measured at smoke-test scale and will differ on the cluster. The reachability figure comes from an offline kinematic probe, not from the simulator, and depends on the layout file at the time of writing.

References

[1] UW Lab. Emergent dexterity via diverse resets and large-scale reinforcement learning. In *International Conference on Learning Representations (ICLR)*, 2026. arXiv:2603.15789.

[2] V.-D. Nguyen. Constructing force-closure grasps. *The International Journal of Robotics Research*, 7(3):3–16, 1988.

[3] M. Mittal, C. Yu, Q. Yu, J. Liu, N. Rudin, D. Hoeller, J. L. Yuan, R. Singh, Y. Guo, H. Mazhar, A. Mandlekar, B. Babich, G. State, M. Hutter, and A. Garg. Orbit: A unified simulation framework for interactive robot learning environments. *IEEE Robotics and Automation Letters*, 8(6):3740–3747, 2023.

[4] S. R. Buss. Introduction to inverse kinematics with Jacobian transpose, pseudoinverse and damped least squares methods. Technical report, University of California, San Diego, 2004. Unpublished technical report.

[5] K. M. Lynch and F. C. Park. *Modern Robotics: Mechanics, Planning, and Control*. Cambridge University Press, 2017.

A Solutions to the Exercises

E1. Because the time-out flag, which is true only when the step counter has reached the episode length, is one of the conjuncts. Every other clause may hold at step 7, but the conjunction is false until step 20. This guarantees that each saved state has survived the complete hold.

E2. The dataset stores finger *positions*, not finger *commands*. The recorder held the gripper closed, so the fingers pressed on the peg. On restore the positions are the same but no command is active until the policy's first action. If that action opens the gripper, or the controller's default target is the open position, the squeeze disappears and the peg drops. The grasp was valid; the policy has to keep it.

E3. At the IK placement, not at the success test. A robot that moves more than $10\,\text{cm}$ during a hold with a zero arm action was placed somewhere physics would not accept, most often beyond a joint limit or in collision. Check the IK print-out for the fraction of environments reaching the target, then whether the placement respects joint limits, then whether the layout is within reach at all.

E4. Because the perturbation models imprecision of the hand, such as a grasp a centimetre further along the fingers or rotated slightly about the approach axis. Those are naturally expressed in the gripper's own axes. The same numbers applied in the object frame would mean something different for every grasp orientation.

E5. When the GPU is saturated. Each environment adds physics bodies, contact pairs and a 1024-point collision cloud per link. Past the point where a single step no longer takes constant time, more environments make each step slower and the gain flattens. The formula also assumes that the success probability is independent of the batch, which holds here because environments do not interact.